

Lab 02 Template

Part 01

- 1) Take a screenshot of your ciphertext character frequencies
(5 points)

```
cp3310@cp3310:~/homework/lab02/part01$ python3 Pulkit_lab02_part1.py ciphertext.txt
Frequency Analysis:
Character      Count    Percentage
R              377     13.25%
B              237     8.33%
Q              210     7.38%
L              199     6.99%
W              197     6.92%
J              185     6.50%
T              179     6.29%
S              178     6.26%
H              172     6.05%
I              149     5.24%
G              138     4.85%
U              81      2.85%
C              71      2.50%
Z              68      2.39%
X              67      2.36%
O              59      2.07%
D              58      2.04%
V              57      2.00%
A              57      2.00%
E              40      1.41%
N              32      1.12%
Y              20      0.70%
M              9       0.32%
K              2       0.07%
F              2       0.07%
P              1       0.04%
```

- 2) Documentation of the iterations to get to the plaintext message. Include a sentence between each iteration explaining the changes.
(15 points)

3) What is the plaintext message?

(10 points)

4) What is the key (the mapping of ciphertext character to plaintext character)?

(10 points)

5) Besides the English language frequency of each character, what else could you have calculated to help you find the plaintext? Explain the difference between the sliding window method and the block method.

(10 points)

Ans) Instead of counting individual letters, we can calculate the bigram frequency, where we calculate the frequency of 2 common English letters together, like IN, A,N etc. or we can calculate the trigram frequency, where we can calculate the frequency of 3 letters together like THE, AND etc., in our ciphertext. In sliding window method, we calculate the frequency by moving from 1 character or letter to the other one and so in this case we are able to cover all 2-letter and 3-letter words and the ones that overlap with each other as well. In the Block method, we basically divide the ciphertext into blocks, and over here, each block has a fixed number of characters and only analyzes the characters in those fixed blocks, and in this case, it ignores the patterns that overlap.

6) Explain the difference between conducting an exhaustive key search vs. English language character frequency. In this lab, which should help you find a solution faster? Explain how your results supported or disproved the number of attempts needed to correctly decrypt the message.

(15 points)

Ans) In exhaustive key search, we try every possible substitution key until the plaintext key is found; for example, in a monoalphabetic substitution cipher, we have 26! Possible keys over here, which is a significant number, and is definitely a time-consuming process. In our lab, I have used English language character frequency because it basically uses the statistical properties of the common English language, such as common letter frequencies (which we have used in this lab) and common words AN, IN, THE, and AND (bigram and trigram) to make our educated guesses and finally figure out the key. Additionally, English language frequency also requires me to do a limited number of iterations (as we can see in Q2) to identify the correct mapping. Hence, my results demonstrate that English-language frequency is faster than exhaustive key search.

7) Find a character frequency distribution for another language. Provide the frequency distribution in your lab report *not a link, but the actual distribution) and be sure to identify which language it is.

(5 points)

Ans) Hindi:

Letter	Frequency
ँ	0.30 %
ं	3.44 %
ः	0.01 %
अ	1.01 %
आ	0.66 %
इ	0.31 %
ई	0.20 %
उ	0.78 %
ऊ	0.05 %
ऋ	< 0.01 %
ए	1.34 %
ऐ	0.03 %
ऑ	0.03 %
ओ	0.10 %
औ	0.94 %
क	7.14 %
ख	0.70 %
ग	1.31 %
घ	0.20 %
च	1.16 %
छ	0.45 %

ज	1.39 %
झ	0.19 %
ञ	0.01 %
ट	0.57 %
ठ	0.17 %
ड	0.75 %
ढ	0.09 %
ण	0.13 %
त	2.89 %
थ	1.15 %
द	1.68 %
ध	0.36 %
न	3.48 %
प	2.66 %
फ	0.35 %
ब	1.78 %
भ	0.67 %
म	3.28 %
य	2.20 %
र	5.91 %
ल	2.45 %
व	1.62 %
श	0.76 %
ष	0.27 %

स	3.78 %
ह	4.82 %
ं	0.67 %
ा	8.22 %
ि	3.20 %
ी	3.47 %
ु	1.45 %
ू	0.81 %
ृ	0.10 %
ँ	< 0.01 %
े	6.85 %
ै	1.96 %
ॉ	0.10 %
ो	2.21 %
ौ	0.15 %
्	3.02 %

ा	8.22 %
क	7.14 %
े	6.85 %
र	5.91 %
ह	4.82 %
स	3.78 %
न	3.48 %

ी	3.47 %
ं	3.44 %
म	3.28 %
ि	3.20 %
्	3.02 %
त	2.89 %
प	2.66 %
ल	2.45 %
ो	2.21 %
य	2.20 %
ै	1.96 %
ब	1.78 %
द	1.68 %
व	1.62 %
ु	1.45 %
ज	1.39 %
ए	1.34 %
ग	1.31 %
च	1.16 %
थ	1.15 %
अ	1.01 %
औ	0.94 %
्र	0.81 %
उ	0.78 %

श	0.76 %
ड	0.75 %
ख	0.70 %
ं	0.67 %
भ	0.67 %
आ	0.66 %
ट	0.57 %
छ	0.45 %
ध	0.36 %
फ	0.35 %
इ	0.31 %
ँ	0.30 %
ष	0.27 %
घ	0.20 %
ई	0.20 %
झ	0.19 %
ठ	0.17 %
ौ	0.15 %
ण	0.13 %
ॉ	0.10 %
ओ	0.10 %
ृ	0.10 %
ढ	0.09 %
ऊ	0.05 %

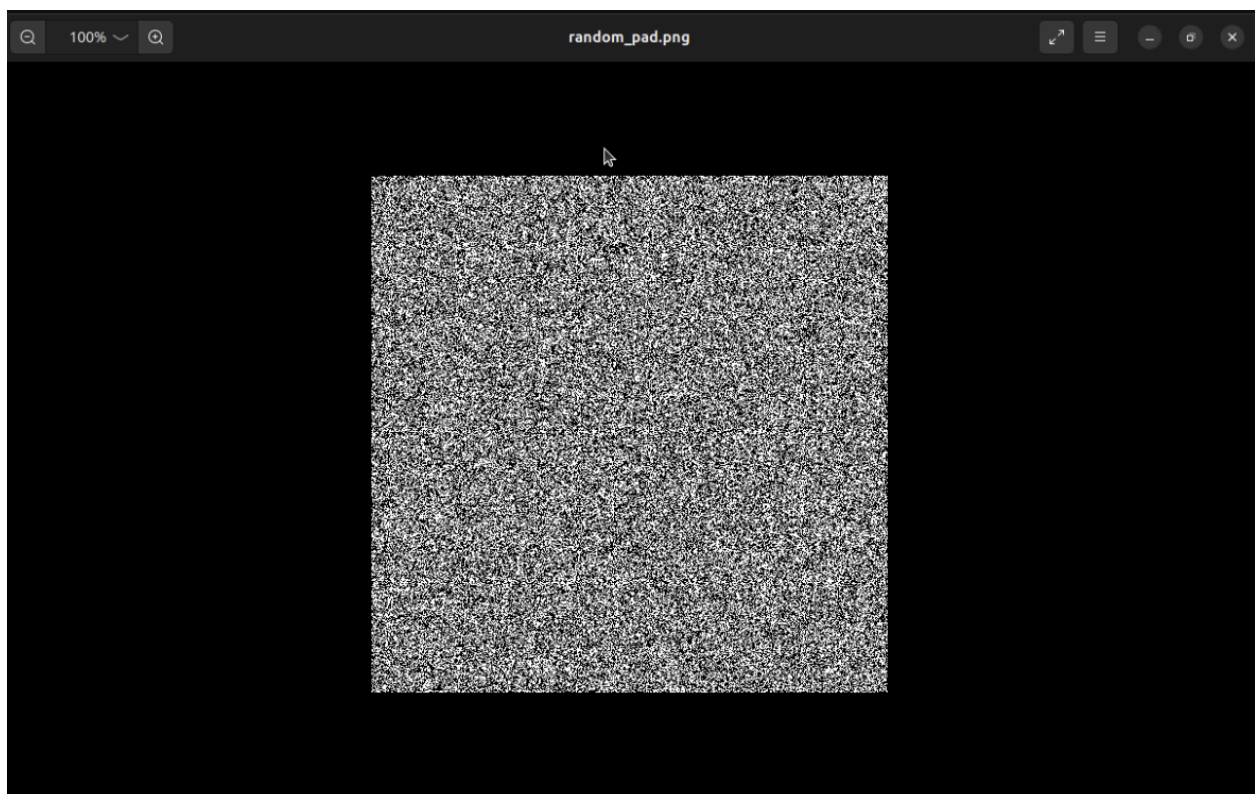
ऐ	0.03 %
आँ	0.03 %
ज	0.01 %
ः	0.01 %
ँ	< 0.01 %
ऋ	< 0.01 %

- 8) **Submit your commented code as `<netid>_lab02_part01.py` to Canvas.**
(10 points)

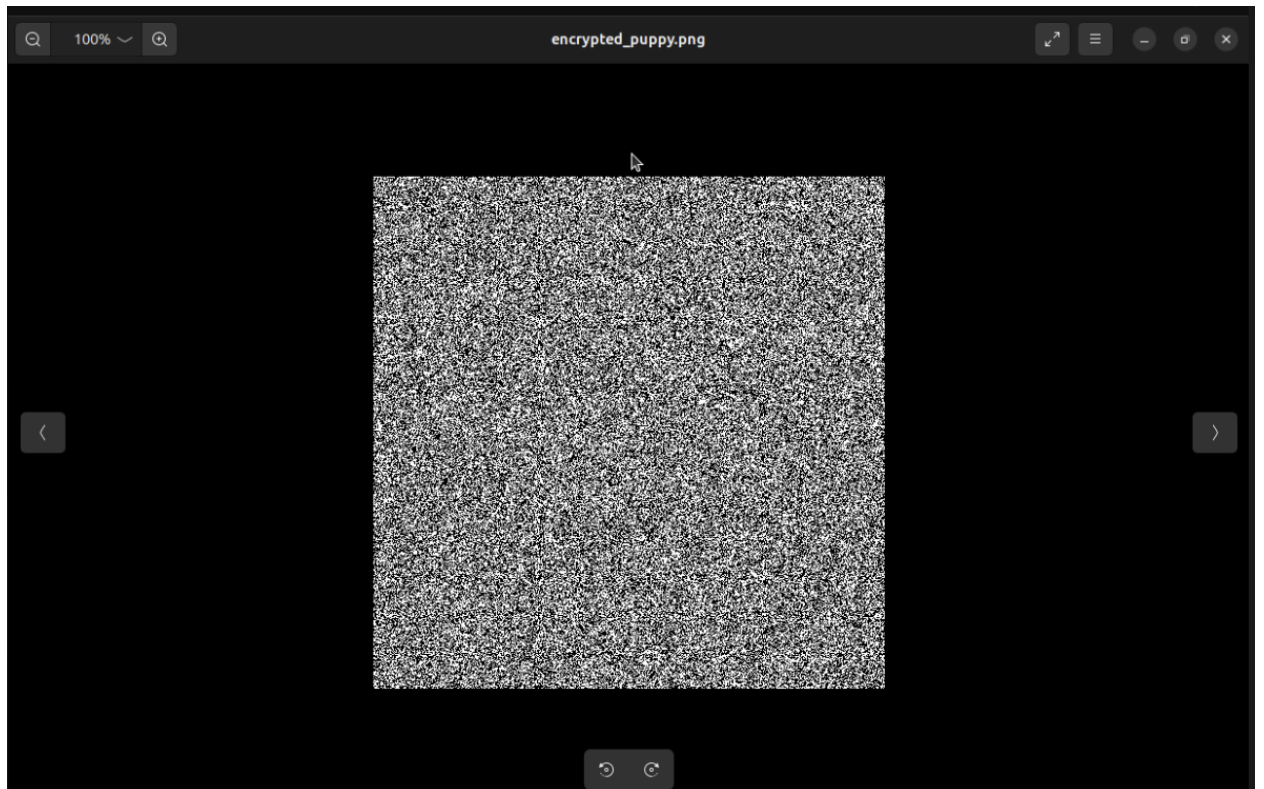
Part 02

- 9) **Include screenshots of `random_pad.png`, `encrypted_puppy.png`, `encrypted_heart.png`, and `leaked_info.png` in your report.**
(10 points)

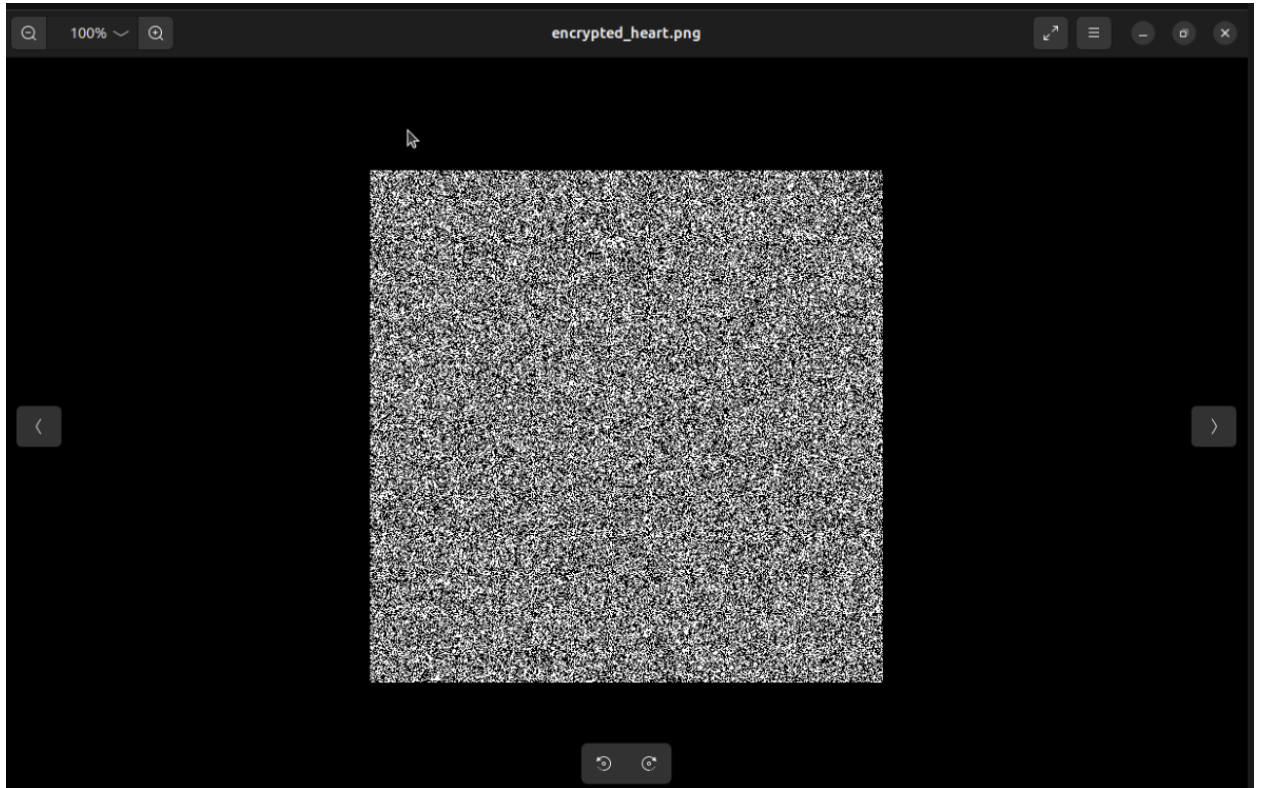
Random_pad.png



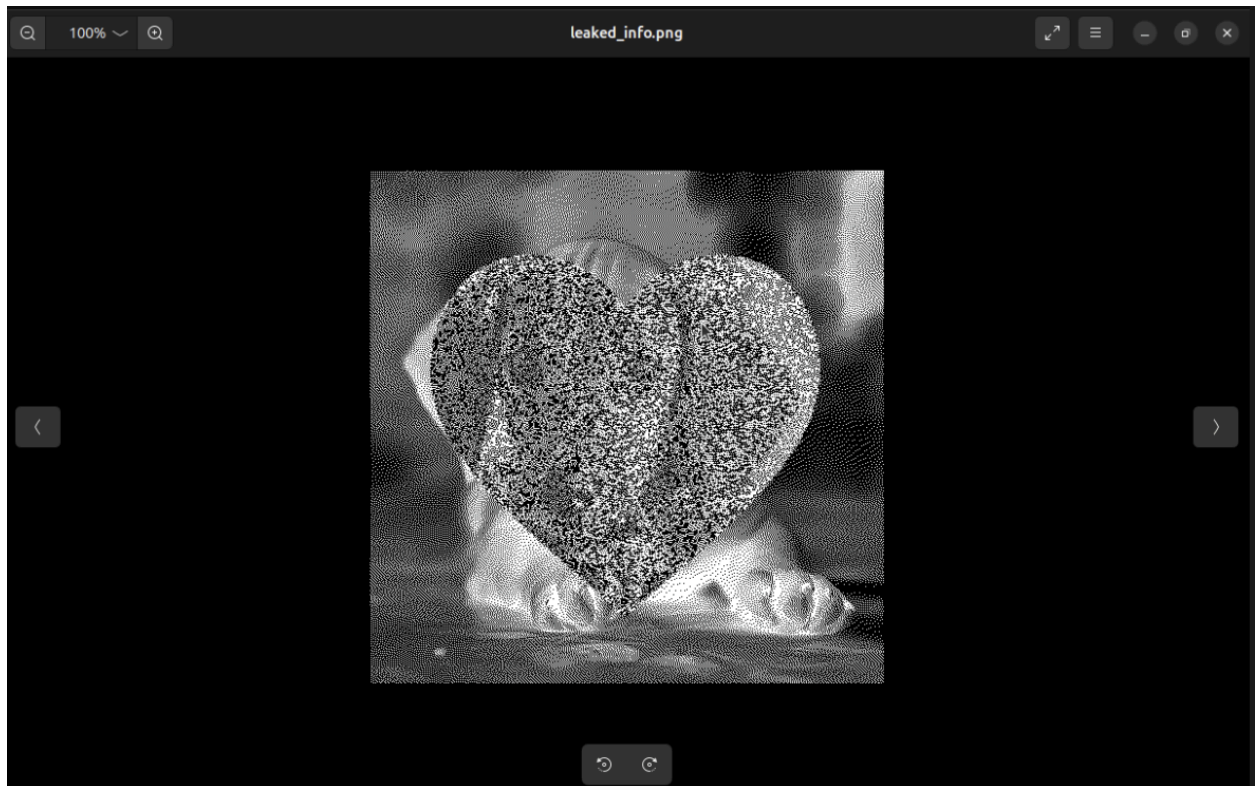
Encrypted_puppy.png



Encrypted_heart.png



Leaked info



10) What do you observe in leaked_info.png? Why does this happen?
(10 points)

Ans) We observe that both images are basically combined right now, that we can see the picture of the heart with the image of a puppy in the background. This happens because both the images were encrypted using the same one-time pad and when 2 images are XORed together, their one time pad cancels out and, since $\text{Plaintext 1} \oplus \text{OTP} \oplus \text{Plaintext 2} \oplus \text{OTP}$ mathematically turns out to be $\text{Plaintext 1} \oplus \text{Plaintext 2}$ since $\text{OTP} \oplus \text{OTP}$ cancels out each other and equals 0, which in turn releases some information about the original pictures as well.